Data Structures - II
Assignment - I

Salma Zeyad Mohamed - 9891
Nada Osama Elsayed - 10047
Abdullah Abdulghany - 10204

1. Objective & Methodology

Objective:

The goal of this lab is to test and understand the difference between the performance of three sorting algorithms: Bubble sort, Selection sort, Insertion sort**.** Specifically, the lab aims to measure the execution time of these algorithms across different input sizes.

Methodology:

For fair and accurate comparison:

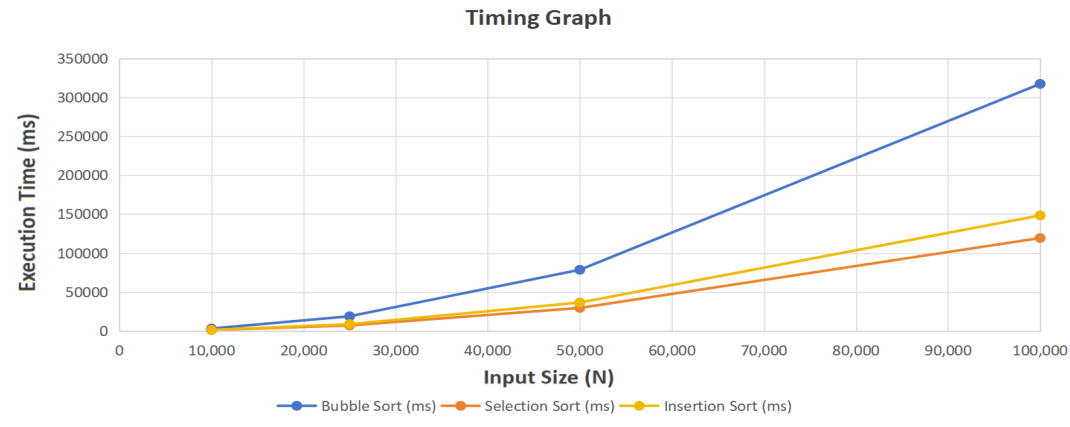
1. Random arrays of integers were generated for specific input sizes: 10,000, 25,000, 50,000, and 100,000 elements.
2. A single random array was generated for each input size and was passed to all three sorting algorithms.

2. Results & Graph

Data Table

Array Size	Bubble sort time(ms)	Insertion sort time(ms)	Selection sort time(ms)
10,000	3069.995	1358.794	1143.995
25,000	18843.377	8665.911	7095.068
50,000	78549.131	36498.789	29549.829
100,000	317407.876	148414.113	119363.366

Graph



Data Summary
It is observed that, as the size of the array increases, the time for all three algorithms increases in a sharp upward curve. Bubble sort's line goes up the higher which means it's the slowest for large array sizes. Selection sort and insertion sort grow much more slowly, with selection sort taking the least amount of time.

3. Performance Analysis

The execution time for all three implemented algorithms exhibits quadratic growth as the array size increases, aligning directly with their $O(n^2)$ sorting classification.
Despite sharing the same Big-O time complexity, the real-world performance varies due to the specific operation mix of each algorithm.

4.Theoretical Comparison

- **Bubble sort** :The worst-case running time is $O(n^2)$ when the array is in reverse order. If optimized with an early-exit flag, its best-case running time is $O(n)$ for an already sorted array.
- **Selection Sort:** Both the best and worst-case running times remain strictly $O(n^2)$ The algorithm must consistently scan the entire unsorted subsection to find the minimum value, making its execution time completely independent of the data's initial arrangement.
- **Insertion Sort:** The worst-case running time is $O(n^2)$. However, it is highly adaptive; its best-case running time is $O(n)$ when handling sorted or partially sorted data.

Because the testing methodology utilized randomly generated arrays passed equally to all algorithms, the recorded execution times reflect the average-case scenario, which scales at $O(n^2)$ across the board.

Conclusion

Even though all three algorithms share an $O(n^2)$ time complexity, they perform differently in the real world because of their specific operational tradeoffs:

- Selection sort:Proved to be the most efficient .By only making one swap per pass,it does the absolute minimum number of memory writes
- Insertion sort:Performed consistently, but slower than Selection Sort.While it shifts elements rather than swapping them.
- Bubble Sort: The easiest to code, but usually the slowest overall because it swaps numbers way too much.